

Personal Finance Analyzer

SQL Project — MySQL 8.0

Name: Srishti Singh

Tool: MySQL Workbench

Database: finance_analyzer

Queries: 6 Analysis Queries

Data: 103 transactions | 2024

Query 1:

USE finance_analyzer;

SELECT

DATE_FORMAT(txn_date, '%Y-%m') AS month,

SUM(CASE WHEN transaction_type = 'income' THEN amount ELSE 0 END) AS total_income,

SUM(CASE WHEN transaction_type = 'expense' THEN amount ELSE 0 END) AS total_expense,

SUM(CASE WHEN transaction_type = 'income' THEN amount ELSE -amount END) AS net_savings

FROM transactions

GROUP BY DATE_FORMAT(txn_date, '%Y-%m')

ORDER BY month;

The screenshot displays the MySQL Workbench interface. The top pane shows the SQL query for Query 1, which uses conditional aggregation to calculate monthly financial metrics. The bottom pane shows the 'Result Grid' with 12 rows of data for the months of 2024. The columns are 'month', 'total_income', 'total_expense', and 'net_savings'. The 'net_savings' column shows values ranging from ₹25,000 in December to ₹66,200 in November. The bottom pane also shows the 'Output' tab with a log of database actions, including table creation and the execution of the query.

month	total_income	total_expense	net_savings
2024-01	87000.00	34400.00	52600.00
2024-02	75000.00	37100.00	37900.00
2024-03	93000.00	24300.00	68600.00
2024-04	79000.00	43800.00	35200.00
2024-05	95000.00	48100.00	46900.00
2024-06	75000.00	39500.00	35500.00
2024-07	105000.00	42900.00	62100.00
2024-08	80000.00	41100.00	38900.00
2024-09	102000.00	45200.00	56800.00
2024-10	80000.00	55000.00	25000.00
2024-11	110000.00	43800.00	66200.00
2024-12	80000.00	55000.00	25000.00

Finding: Across all 12 months of 2024, income consistently exceeded expenses every single month showing a healthy savings pattern. The highest savings month was November (₹66,200) when income peaked at ₹1,10,000. The lowest savings month was December (₹25,000) despite decent income of ₹80,000, suggesting unusually high spending that month. Average monthly savings across the year was approximately ₹43,800.

Query 2:

```
USE finance_analyzer;
```

```
SELECT c.category_name, SUM(t.amount) AS total_spent
```

```
FROM transactions t
```

```
JOIN categories c ON t.category_id = c.category_id
```

```
WHERE t.transaction_type = 'expense'
```

```
GROUP BY c.category_name
```

```
ORDER BY total_spent DESC
```

```
LIMIT 5;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
1. USE finance_analyzer;
2. SELECT c.category_name, SUM(t.amount) AS total_spent
3. FROM transactions t
4. JOIN categories c ON t.category_id = c.category_id
5. WHERE t.transaction_type = 'expense'
6. GROUP BY c.category_name
7. ORDER BY total_spent DESC
```

The Results tab displays the following data:

category_name	total_spent
Rent	240000.00
Shopping	71800.00
Entertainment	51600.00
Investment	50000.00
Groceries	41300.00

The bottom status bar shows: "1 08:48:09 SELECT c.category_name, SUM(t.amount) AS total_spent FROM transactions t JOIN categories c ON t.category_id ... 5 row(s) returned".

Finding: Rent is the biggest expense at ₹2,40,000, making up nearly 46% of total annual expenses. Shopping (₹71,800) and Entertainment (₹51,600) are the next biggest drains, suggesting areas to cut back for better savings.

Query 3:

USE finance_analyzer;

WITH monthly AS (

SELECT DATE_FORMAT(txn_date,'%Y-%m') AS month,

SUM(amount) AS total_expense

FROM transactions WHERE transaction_type = 'expense'

GROUP BY DATE_FORMAT(txn_date,'%Y-%m')

)

SELECT month, total_expense,

LAG(total_expense) OVER (ORDER BY month) AS prev_month,

total_expense - LAG(total_expense) OVER (ORDER BY month) AS change_amt

FROM monthly;

The screenshot shows the MySQL Workbench interface. The 'Query 1' tab is active, displaying the following SQL query:

```
6 GROUP BY DATE_FORMAT(txn_date,'%Y-%m')
7 )
8 SELECT month, total_expense,
9 LAG(total_expense) OVER (ORDER BY month) AS prev_month,
10 total_expense - LAG(total_expense) OVER (ORDER BY month) AS change_amt
11 FROM monthly;
```

The 'Result Grid' shows the following data:

month	total_expense	prev_month	change_amt
2024-01	34400.00		
2024-02	37100.00	34400.00	2700.00
2024-03	34100.00	37100.00	-3000.00
2024-04	43500.00	34100.00	9400.00
2024-05	48100.00	43500.00	4600.00
2024-06	39500.00	48100.00	-8600.00
2024-07	42900.00	39500.00	3400.00
2024-08	41100.00	42900.00	-1800.00
2024-09	45200.00	41100.00	4100.00
2024-10	55000.00	45200.00	9800.00
2024-11	43800.00	55000.00	-11200.00
2024-12	55100.00	43800.00	11300.00

The 'Action Output' tab shows the following messages:

#	Time	Action	Message
1	08:48:09	SELECT c.category_name, SUM(t.amount) AS total_spent FROM transactions t JOIN categories c ON t.category_id ...	5 row(s) returned
2	08:51:21	WITH monthly AS (SELECT DATE_FORMAT(txn_date,'%Y-%m') AS month, SUM(amount) AS total_expense FR...	12 row(s) returned

Finding: Expenses spiked most in April (+₹9,400) and October (+₹9,800). June had the biggest drop (-₹8,600) showing good cost control. December saw a ₹11,300 jump likely festive season spending.

Query 4:

```
USE finance_analyzer;
```

```
SELECT
```

```
    c.category_name,
```

```
    SUM(b.allocated) AS total_budget,
```

```
    SUM(t.amount) AS actual_spent,
```

```
    SUM(b.allocated) - SUM(t.amount) AS remaining
```

```
FROM categories c
```

```
JOIN transactions t ON t.category_id = c.category_id
```

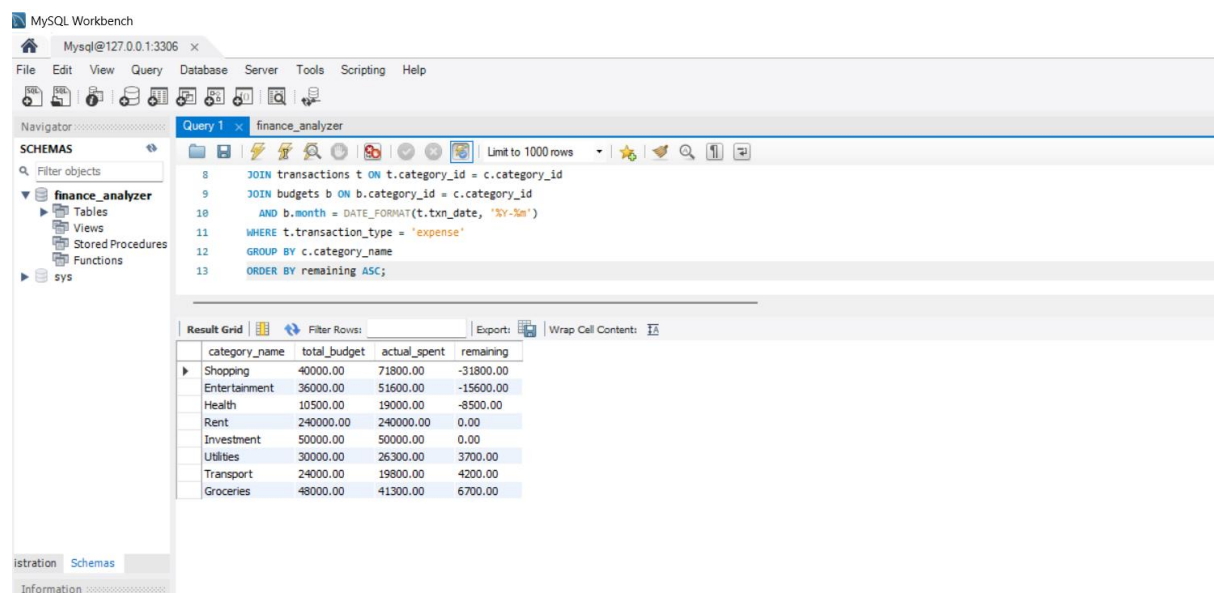
```
JOIN budgets b ON b.category_id = c.category_id
```

```
    AND b.month = DATE_FORMAT(t.txn_date, '%Y-%m')
```

```
WHERE t.transaction_type = 'expense'
```

```
GROUP BY c.category_name
```

```
ORDER BY remaining ASC;
```



MySQL Workbench

Query 1 x finance_analyzer

```
8 JOIN transactions t ON t.category_id = c.category_id
9 JOIN budgets b ON b.category_id = c.category_id
10 AND b.month = DATE_FORMAT(t.txn_date, '%Y-%m')
11 WHERE t.transaction_type = 'expense'
12 GROUP BY c.category_name
13 ORDER BY remaining ASC;
```

category_name	total_budget	actual_spent	remaining
Shopping	40000.00	71800.00	-31800.00
Entertainment	36000.00	51600.00	-15600.00
Health	10500.00	19000.00	-8500.00
Rent	240000.00	240000.00	0.00
Investment	50000.00	50000.00	0.00
Utilities	30000.00	26300.00	3700.00
Transport	24000.00	19800.00	4200.00
Groceries	48000.00	41300.00	6700.00

Finding: Shopping overspent by ₹31,800 (79% over budget) and Entertainment by ₹15,600. Groceries and Transport stayed under budget. Rent was spent exactly as budgeted at ₹2,40,000.

Query 5:

```
USE finance_analyzer;
```

```
SELECT txn_date, description, amount
```

```
FROM transactions
```

```
WHERE transaction_type = 'expense'
```

```
ORDER BY amount DESC
```

```
LIMIT 10;
```

The screenshot shows the MySQL Workbench interface. The 'Query Editor' window displays the following SQL query:

```
1 USE finance_analyzer;
2 SELECT txn_date, description, amount
3 FROM transactions
4 WHERE transaction_type = 'expense'
5 ORDER BY amount DESC
6 LIMIT 10;
```

The 'Result Grid' window shows the results of the query, displaying 10 rows of data. The columns are 'txn_date', 'description', and 'amount'. All 10 rows show 'House Rent' payments of ₹20,000.00, occurring at regular intervals throughout 2024.

txn_date	description	amount
2024-01-03	House Rent	20000.00
2024-02-03	House Rent	20000.00
2024-03-03	House Rent	20000.00
2024-04-03	House Rent	20000.00
2024-05-03	House Rent	20000.00
2024-06-03	House Rent	20000.00
2024-07-03	House Rent	20000.00
2024-08-03	House Rent	20000.00
2024-09-03	House Rent	20000.00
2024-10-03	House Rent	20000.00

Finding: All top 10 largest single expenses are House Rent payments of ₹20,000 each, paid consistently every month throughout 2024 confirming it as the biggest fixed expense.

Query 6 :

USE finance_analyzer;

SELECT

transaction_type,

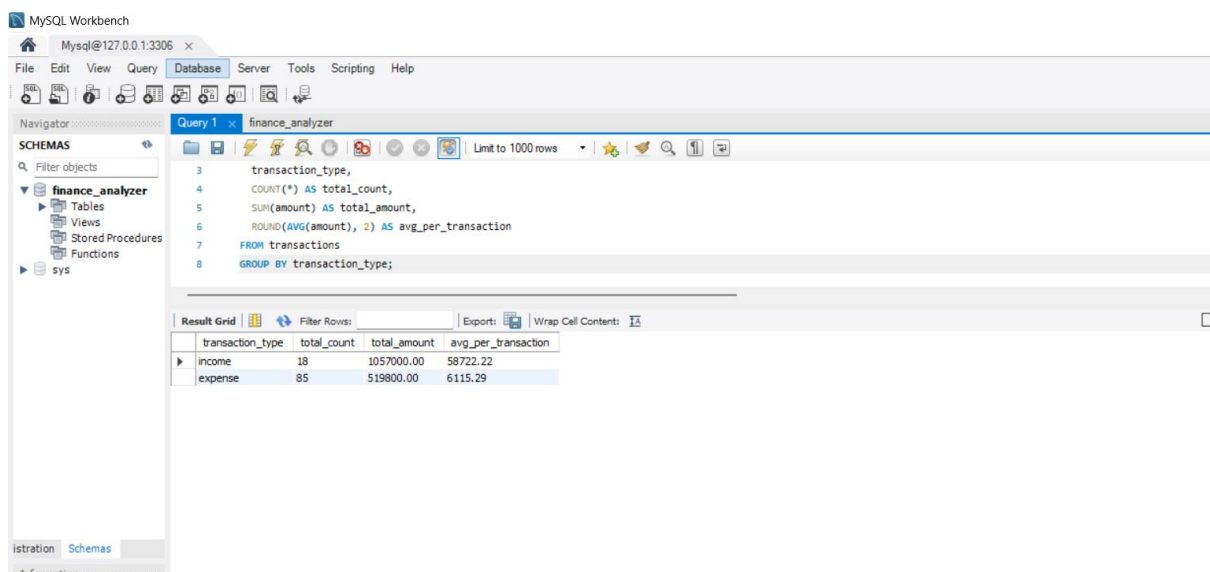
COUNT(*) AS total_count,

SUM(amount) AS total_amount,

ROUND(AVG(amount), 2) AS avg_per_transaction

FROM transactions

GROUP BY transaction_type;



MySQL Workbench

Query 1: finance_analyzer

```
3 transaction_type,
4 COUNT(*) AS total_count,
5 SUM(amount) AS total_amount,
6 ROUND(AVG(amount), 2) AS avg_per_transaction
7 FROM transactions
8 GROUP BY transaction_type;
```

transaction_type	total_count	total_amount	avg_per_transaction
income	18	1057000.00	58722.22
expense	85	519800.00	6115.29
	103	1576800.00	15211.11

Finding: Total income of ₹10,57,000 significantly exceeded total expenses of ₹5,19,800 resulting in annual net savings of ₹5,37,200 (50.8% savings rate). There were 85 expense transactions vs only 18 income transactions, with average income per transaction being 9.6x higher.